CSC 212: Data Structures and Abstractions

10: Recursion

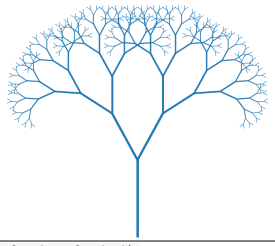
Prof. Marco Alvarez

Department of Computer Science and Statistics
University of Rhode Island

Fall 2026



A tree is a trunk with two smaller trees



```
def draw(x, y, length, angle, depth):
    if depth == 0:
        return                                  # base case
    end_x = x + length * math.cos(angle)
    end_y = y + length * math.sin(angle)
    plt.plot([x, end_x], [y, end_y])            # the trunk
    draw(end_x, end_y, length * 0.7, angle + 0.5, depth - 1) # left tree
    draw(end_x, end_y, length * 0.7, angle - 0.5, depth - 1) # right tree

draw(x=0, y=0, length=1, angle=math.pi/2, depth=9)
```


Q10-1

- How many segments does `draw(..., depth=9)` plot?

```
def draw(x, y, length, angle, depth):
    if depth == 0:
        return                                  # base case
    end_x = x + length * math.cos(angle)
    end_y = y + length * math.sin(angle)
    plt.plot([x, end_x], [y, end_y])            # the trunk
    draw(end_x, end_y, length * 0.7, angle + 0.5, depth - 1) # left tree
    draw(end_x, end_y, length * 0.7, angle - 0.5, depth - 1) # right tree

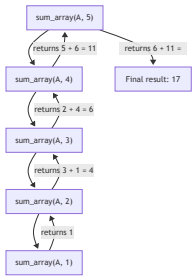
draw(x=0, y=0, length=1, angle=math.pi/2, depth=9)
```

A. 9
B. 18
C. 511
D. 1023
E. None of the above

Recursion

- A recursion function has:
  - ✓ a **base case**: an input small enough to answer directly
  - ✓ a **recursive case**: solve the input by calling itself on a **smaller** input
- Every call must make **progress** toward a base case

A = {1, 3, 2, 5, 6}




```
int sum_array(const std::vector<int>& A, int n) {
    if (n == 1)
        return A[0];
    int s = sum_array(A, n - 1);
    return A[n - 1] + s
}
```

Q10-2

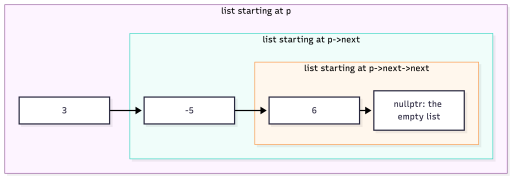
- The function `sum_array` expects $n \geq 1$. What does `sum_array(A, 0)` do, with `A = {1,3,2,5,6}`?

```
int sum_array(const std::vector<int>& A, int n) {
    if (n == 1)
        return A[0];
    int s = sum_array(A, n - 1);
    return A[n - 1] + s
}
```

A. Returns 0
B. Returns 17
C. Throws `std::out_of_range`
D. Crashes
E. None of the above

Linked lists: recursive definition

- A linked list is either:
  - ✓ empty, or
  - ✓ a node, followed by a smaller linked list starting at `p->next`



Recursive methods

```
size_t SLList::size_r(Node* p) const {
    if (p == nullptr)              // empty list
        return 0;
    return 1 + size_r(p->next);    // this node + the rest
}

bool SLList::contains_r(Node* p, int value) const {
    if (p == nullptr)
        return false;              // nothing left to search
    if (p->data == value)
        return true;
    return contains_r(p->next, value);
}
```

Solve one node checking that this step is right, and trust the recursive call with the rest.

Wrappers and helpers

- Recursive methods for linked lists need a `Node*` parameter
  - ✓ however, users of the class must never see a `Node*`
- So every recursive method comes in two parts:
  - ✓ a public wrapper, with the clean interface
  - ✓ a private helper, that does the recursion

```
class SLList {
    private:
        // …
        // recursive helpers
        size_t size_r(Node* p) const;
        bool contains_r(Node* p, int value) const;

    public:
        // ...
        // public wrappers
        size_t size() const { return size_r(m_head); }
        bool contains(int v) const { return contains_r(m_head, v); }
};
```

Q10-3

- The list holds 1 2 3. What does `foo(m_head)` print?

```
void foo(Node* p) {
    if (p == nullptr)
        return;
    std::cout << p->data;
    foo(p->next);
    std::cout << p->data;
}
```

A. 123
B. 112233
C. 123321
D. 321123
E. None of the above

Clear a list recursively

```
void SLList::clear_r(Node* p) {
    if (p == nullptr)
        return;
    clear_r(p->next);           // free the rest first
    delete p;                   // p->next is no longer needed
}

void SLList::clear() {
    clear_r(m_head);
    m_head = m_tail = nullptr;  // reset the list
    m_size = 0;
}
```


Q10-4

- A student writes `clear_r` with the two lines swapped. What is wrong with it?

```
void clear_r(Node* p) {
    if (p == nullptr)
        return;
    delete p;
    clear_r(p->next);
}
```

A. It reads `p->next` after `delete p`: undefined behavior
B. Nothing: it frees the same nodes in the other order
C. It frees only the first node
D. It never reaches the base case
E. None of the above

Changing the list: return the new rest

- To **change** a list recursively, the helper returns the first node of the list **after** the change
  - ✓ nodes that did not change get the same pointer back

```
SLList::Node* SLList::remove_r(Node* p, int value) {
    if (p == nullptr)
        return nullptr;                 // not found: nothing changes
    if (p->data == value) {
        Node* rest = p->next;
        delete p;
        --m_size;
        return rest;                    // the list without p
    }
    p->next = remove_r(p->next, value);   // the new rest, linked in
    if (p->next == nullptr) m_tail = p;   // on the way back: update tail if needed
    return p;
}

void SLList::remove(int value) {          // removes the first node holding value
    m_head = remove_r(m_head, value);     // the head may change too
    if (m_head == nullptr)
        m_tail = nullptr;                 // the list is now empty
}
```

*The tail is fixed **on the way back**: the first node whose* `next` *is now* `nullptr` *is the last node.*

Reverse a list recursively

```
SLList::Node* SLList::reverse_r(Node* p) {
    if (p == nullptr || p->next == nullptr)
        return p;                   // 0 or 1 node: already reversed
    Node* new_head = reverse_r(p->next); // reverse the rest
    p->next->next = p;              // the old second node points back to p
    p->next = nullptr;              // p is now the last node
    return new_head;
}

void SLList::reverse() {
    m_tail = m_head;
    m_head = reverse_r(m_head);
}
```

trace algorithm manually with: `1 -> 2 -> 3 -> 4`

Q10-5

- Delete the line `p->next = nullptr` from `reverse_r`. The list is `1 2 3`. What does `print()` show after `reverse_r()`?

```
SLList::Node* SLList::reverse_r(Node* p) {
    if (p == nullptr || p->next == nullptr)
        return p;                   // 0 or 1 node: already reversed
    Node* new_head = reverse_r(p->next); // reverse the rest
    p->next->next = p;              // the old second node points back to p
    p->next = nullptr;              // p is now the last node
    return new_head;
}
```

A. 3 2 1

B. 3 2

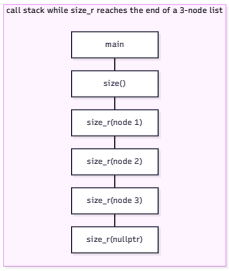
C. 3 2 1 2 1 2 1 ... forever

D. 1 2 3

E. None of the above

How deep can the recursion go?

- Each pending call holds a frame on the stack until its recursive call returns
  - ✓ `size_r` on a list of n nodes uses $n+1$ frames at the deepest point
- A loop over a list needs $\Theta(1)$ extra space; the recursive version needs $\Theta(n)$

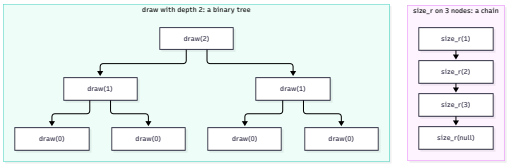



```
size_t SLList::size_r(Node* p) const {
    if (p == nullptr)
        return 0;
    return 1 + size_r(p->next);
}
```


Recursion call trees

Recursion call tree

- A recursion call tree draws calls made after a top-level call
  - the **root** is the first call
  - each **node** is one call; its children are the calls it makes
  - the **leaves** are base cases
  - the **height** is the deepest chain of pending calls: the stack depth
  - the **number of nodes** is the total number of calls



Exercises

- Write recursive versions of `int& at(size_t index)` and `int sum() const` for `SLList`
  - `at`: the element at `index` of the list starting at `p` is `p->data` when `index == 0`, and otherwise the element at `index - 1` of the rest
  - `sum` of an empty list is 0
- Write `int max() const` recursively, calling the helper once per node
  - throw `std::underflow_error` for an empty list
- Write `size_t count(int value) const`: how many nodes hold `value`?
- Write `bool equals(const SLList& other) const`: returns true if same length, same values, same order
  - hint: the helper takes two node pointers and walks both lists together
- Write `void insert_sorted(int value)` for a list kept in increasing order
  - assume a list 1 4 9, `insert_sorted(5)` gives 1 4 5 9
  - do not forget `m_tail` and `m_size`
- Trace `print_back_r` on 3 -5 6 18 by drawing its call tree
  - number each `std::cout` in the order it runs
  - how many calls are made? what is the largest number of frames on the stack at once?